Cryptographic Hash Functions - Modern hash function analysis

Elek Andreea

Cybersecurity, Year 1.

Contributing authors: andreea.elek00@e-uvt.ro;

Abstract

This paper focuses on five modern implementations of hash functions: SHA-256, SHA-3, Whirlpool, BLAKE2b and RIPEMD-160. While all five are used to obtain hash functions, with the purpose of proving and verifying data integrity, the five hash function have some core differences regarding the algorithms they use and the areas of cybersecurity where they are most often utilized. SHA-256 uses the Merkle-Damgard algorithm and is utilized for Standard file integrity and Bitcoin block mining. SHA-3 uses the Sponge construction and in an alternative to SHA-2, being immune to lenght-extension. The Whirlpool hash function is AES-based and uses high-security block cipher logic. BLAKE2b is ChaCha-based and it is optimized for speed, often being used in Linux and WinRAR. RIPEM-160 is another hash function based on the Markle-Damgard, which is famously used in Bitcoin's address generation. For this paper, a rigorous study was conducted to analyze technical parameters such as data processing rates and time consumption for all of the five mentioned hash functions.

Keywords: hash, functions, SHA, RIPEMD, BLAKE2b, Whirlpool

1 Introduction

The main purpose of hash functions is to ensure data integrity. A hash function acts as a "digital fingerprint", a unique identifier for a piece of data. In cybersecurity, it ensures that a message, file, or password has not been altered or corrupted.

The hashes are the output of a hashing algorithm, which produces a unique, fixed-length string – the hash value, or "message digest" – for any given piece of data or "message". [1]

A hash function ensures integrity through its deterministic nature: the same input will always produce the same output. If even a single bit of the input data is changed, the resulting hash will change completely. This way, the integrity is verified by comparing a newly calculated hash against a known "good" hash. If they match, the data was not tampered with. [2] Security is also maintained because hashes are irreversible (Pre-image Resistance). You cannot "reverse-engineer" a file from its hash. [3]

This type of function is a one-way function, which means that it allows systems to verify the integrity of information without actually seeing or storing the sensitive original data. [4] Another important propriety of hash functions is their resistance to collision, which means that it is highly unlikely that two different inputs would produce the same hash value. In other words, the chances of two different messages having the same hash value are almost null. [5]

While there are multiple hash functions that essentially do the same think, it is essential to note that they have different speeds and different security levels, which is why it is important to determine the advantages and disadvantages of the algorithms, and when each one would be best used.

2 Background/Related Work

The basic concept of "hashing" was invented by Hans Peter Luhn at IBM in 1953 with the purpose of organizing information in the computer memory in a way which would allow it to be quickly and easily found when needed. [6] In the late 1970s, the "speed tool" became a "security tool." Two major milestones defined this era: in 1976, Diffie and Hellman identified the need for a "one-way" function, then, 1979, Ralph Merkle invented the Merkle Tree and formally described the requirements for a cryptographic hash function, specifically that it must be "collision-resistant". [7]

Since then, multiple hash algorithms have been developed. Five of those algorithms are those analyzed in this paper: SHA-256 , SHA-3, Whirlpool , BLAKE2b, RIPEMD-160. One very important thing to note about these algorithms is that the hashes produced for the same string sequence by them will be different for each one, as each hash algorithm has a different implementation logic.

Modern hashes are known to be categorized by their internal "blueprint". Merkle-Damgård, which is the most common design, is used by both SHA-256 and RIPEMD-160. This algorithm breaks data into blocks and processes them in a chain. [8]

The Sponge Construction is used by SHA-3, and it "absorbs" data into a state and "squeezes" it out, offering high security against modern attacks. [9] AES-based hashing, which is used for Whirlpool, uses the math of block ciphers (encryption) to create hashes, meaning that it takes an arbitrary bit string as input and then returns a fixed length string as output. [10]

BLAKE2b is based on the ChaCha20 stream cipher core, which is known as a highly-optimized permutation-based design. It was created to be significantly faster than SHA-3 (and even SHA-256) on modern 64-bit CPUs, by using a design that avoids the complexity of older standards. [11]

3 Methodology

The first thing that was done was to implement all 5 algorithms, using the C programming language and standard libraries. The first test was performed to verify the correctness of the implementation for each algorithm. This was done by checking the hash result for each algorithm multiple times, to confirm that the result was the same every time. We also verified the obtained result against the result obtained by existing implementations and already known values for the same string of text.

Fig. 1
Testing correctness of hash for SHA-256

Once I confirmed that the algorithms were implemented correctly, we tested the speed and throughout for 3 different files with sizes of 1KB, 10MB and 100MB with each implementation, 100 times for each file size. For each implementation we also analyzed the mean(1) and median(2) values and the standard deviation(3). The mean value the sum of all the values in the data set divided by the number of values in the data set, also known as the arithmetic average. The median is the middle value for sorted data. Standard deviation measures how much the execution times vary from the mean.

4 Implementation Analysis

4.1 SHA-256

The implementation is built for the OpenSSL 3.x standard. It was written in C and the implementation relies on the libcrypto library and the openssl/evp.h header. The source is available in the OpenSSL GitHub repository: https://github.com/openssl/openssl

4.1.1 Architecture and design

The core logic follows the Merkle-Damgard construction, utilizing a 256-bit internal state processed in 512-bit blocks. This architecture makes it a versatile tool for high-integrity environments, specifically:

- Blockchains: Powering Bitcoin mining and Merkle tree verification.
- Authentication: Generating ECDSA digital signatures.
- Security: Validating file integrity and checksums.

4.1.2 Security profile

With a fixed 256-bit hash length, the implementation offers robust protection:

- Collision Resistance: 128-bit security level.
- Pre-image Resistance: 256-bit security level.

4.2 SHA-3

This SHA3-256 implementation is built in C using the OpenSSL 3.x framework. It requires the openssl/evp.h header and the standard OpenSSL libcrypto provider for execution.

4.2.1 Architecture and design

Unlike traditional Merkle-Damgard functions, this version utilizes a Sponge Construction that functions through a two-phase process: Absorbing: Data is integrated into the internal state. Squeezing: The final hash is extracted from that state.

This specific architecture provides native immunity against length-extension attacks, a significant upgrade over older hashing standards.

4.2.2 Application and security

The implementation can be used for high-security environments, including government-grade data integrity, secure message authentication (MACs), and modern digital signature schemes.

We have an output of 256-bits a high security margin to resist both classical and quantum-level collision searches.

4.3 Whirlpool

For this implementation of Whirlpool, we need to use the OpenSSL 3.x Legacy module. Because Whirlpool is now categorized under legacy standards, the C source must explicitly load the legacy.dll provider to function within the OpenSSL 3.x environment.

4.3.1 Architecture and design

The design is fundamentally AES-based, and uses the mathematical principles of the Square block cipher, and a modified Davies-Meyer construction, which repurposes the block cipher logic into a dedicated hashing function.

4.3.2 Use Cases and Reliability

Whirlpool remains a primary choice for projects requiring ISO/IEC 10118-3 compliance. It is particularly effective for:

- Archival Security: Long-term data protection where stability is key.
- High-End Integrity: Systems that demand a larger security margin than standard 256-bit hashes.

4.3.3 Security Profile

With a massive 512-bit digest size, the algorithm provides an exceptionally high margin for collision resistance, making it robust against brute-force attacks in sensitive data environments.

4.4 BLAKE2b

This implementation of BLAKE2b is built in C using the OpenSSL 3.x libcrypto library. It uses the standard OpenSSL headers, which makes it a good choice for modern cryptography.

4.4.1 Performance and Architecture

The algorithm's core logic is derived from the ChaCha stream cipher. It is specifically optimized for 64-bit architectures, where it typically outperforms SHA-3 in raw speed without sacrificing security.

4.4.2 Application area

Due to its efficiency and robustness, BLAKE2b is used in different areas for high-performance computing, including:

- Storage & Integrity: it is used extensively for cloud storage verification and software checksums (especially for Linux kernel and WinRAR).
- Credential Security: it serves as the fundamental building block for the Argon2 password hashing standard.

4.4.3 Security specifications

Regarding the security specification, we have the following parameters:

- Digest Length: 512-bit output.
- Resilience: Designed to be immune to length-extension attacks and remains highly resistant to all current forms of cryptanalysis.

4.5 RIPEMD

This implementation of RIPEMD-160 operates through the OpenSSL 3.x Legacy Module. Because it is no longer part of the default provider set, I had to explicitly initialize the module using OSSL_PROVIDER_load within the C source code to access the algorithm.

4.5.1 Design and implementation logic

The algorithm is a specialized Merkle-Damgard construction, which uses two parallel streams of processing. This dual-path approach adds a layer of complexity to the transformation process, resulting in an enhanced hash robustness compared to its predecessors.

4.5.2 Applications

While newer hash functions exist, RIPEMD remains a modern and critical standard in specific cryptographic ecosystems:

- Cryptocurrency: it is a core component in generating compressed Bitcoin addresses.
- Encryption Standards: it is used frequently within PGP (Pretty Good Privacy) for digital signatures.
- Legacy Support: it was riginally designed as a secure, 32-bit optimized successor to the MD4 and MD5 families.

4.5.3 Security specifications

This implementation has the following specifications:

- Digest Size: 160-bit.
- Architecture: Optimized for high performance on 32-bit systems.
- Collision Resistance: Provides a significantly higher security margin than MD5, though it is generally paired with other hashes in modern protocols.

5 Experimental Results

5.1 SHA-256

For the data correctness, I verified that the hash for the string "abc" that is produced by the algorithm is the same one that is generated known for this string using a python script. The result showed that the produced hash was the expected one, particularly "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad".

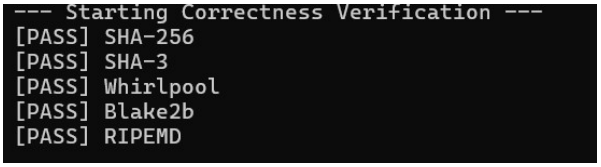

Fig. 2
Hash correctness verification

I performed the hashing operation 100 times on each file size (small, medium, large) using a python script. Analyzing all the results obtained, the average results for the time the hashing took and the throughput are the ones documented in the table below.

	Throughput	Average time (mean)
Small (1KB)	0.06 MB/s	29.23 ms
Medium (10MB)	568.83 MS/s	17.65 ms
Large (100MB)	5,762.83	17.39 ms

Table 1

Throughput and average time for SHA-256

SHA-256 showed exceptional efficiency as the data size increased, jumping from 568 MB/s to over 5.7 GB/s. This indicates a highly optimized implementation likely leveraging hardware acceleration features.

Regarding the median values, we have the following for the three types of files, from small to large: 17.22 ms, 17.25 ms, 17.20 ms. The standard deviation values for the three files are: 118.05 ms, 1.22 ms, 0.84 ms.

For the small 1KB file, SHA-256 exhibited a high Standard Deviation (118.05 ms), primarily due to a startup "lag" in the very first iteration. However, once the environment was "warm," the execution time dropped significantly, to approximatively 16 ms.

5.2 SHA-3

Like in the previous implementation, the same python script showed that the known hash "3a985da74fe225b2045c172d6bd390bd855f086e3e9d525b46bfe24511431532" for the same string "abc" was in fact produced by the implementation for SHA-3.

The mean values (average time of hashing) I obtained for each of the three types of files, from small to large, are: 14.23 ms, 14.37 ms, 14.88 ms. The median values are: 14.09 ms, 14.26 ms, 14.5 ms. The standard deviation values for the three files are: 0.58 ms, 0.49 ms, 1.27 ms. Regarding the throughput, the values we have are 0.07 MB/s for the small 1KB file, 696.38 MB/s for 10 MB and 6,754.36 MB/s for 100 MB.

The result show that SHA-3 has an overall great performance It processed 100MB of data in nearly the same amount of time it took to process 1KB, meaning the actual hashing math is nearly instantaneous.

It is also a very stable implementation, showing very tight consistency (0.49 ms standard deviation) for 10MB files.

5.3 Whirlpool

The data correctness was validated for the Whirlpool implementation by the same python script mentioned in the first two implementations, the hash string produces and expected being "4e2448a4c6f486bb16b6562c73b4020bf3043e3a731bce721ae1b303d97e6d4c7181eebdb6c57e277d0e34957114cbd6c797fc9d95d8b582d225292076d4eef5".

The values obtained during the testing for the Whirlpool implementation, for the small, medium and large file accordingly, are: 18.82 ms, 18.11 ms, 18.17 ms for mean value; 17.86 ms, 17.92 ms, 17.99 ms for median value; 9.35 ms, 0.83 ms, 0.71 ms for standard deviation; 0.06 MB/s, 553.19 MB/s, 5,511.96 MB/s for throughput.

We can see that whirlpool maintains a very steady throughput. It also has a baseline latency of roughly 18 ms regardless of file size, indicating a very predictable performance curve suitable for systems requiring high reliability.

5.4 BLAKE2b

The data correctness test was passed in the case of the BLAKE2b algorithem, the python script validating that the hash produced by the implementation was the same one that was expected for the "abc" string, "ba80a53f981c4d0d6a2797b69f12f6e94c212f14685ac4b74b12bb6fdbffa2d17d87c5392aab792dc252d5de4533cc9518d38aa8dbf1925ab92386edd4009923".

For this implementation we have the following values for the small, medium and large files: 18.18 ms, 17.98 ms, 17.85 ms for mean value; 17.59 ms, 17.85 ms, 17.72 ms for median value; 4.77 ms, 0.87 ms, 0.67 ms for standard deviation; 0.06 MB/s, 557.31 MB/s, 5,608.66 MB/s for throughput.

Blake2b shows a slight edge in speed for 100MB files. It processed the large file consistently around the 17.85 ms mark. Its throughput increased nearly exactly 10-fold when moving from 10MB to 100MB, proving that the implementation is perfectly linear and memory-efficient.

5.5 RIPEMD

Similarly to the other implementations the expected hash result was produced by the function, in this case the hash being "8eb208f7e05d987a9b044a8e98c6b087f15a0bfc".

With the RIPEMD implementation, the following values were obtained for the small, medium and large files, accordingly: 18.50 ms, 18.31 ms, 18.40 ms for mean value; 17.94 ms, 18.12 ms, 18.20 ms for median value; 4.65 ms, 0.79 ms, 0.79 ms for standard deviation; 0.06 MB/s, 547.04 MB/s, 5,443.76 MB/s for throughput.

While RIPEMD appears to be somewhat slow, it still achieves over 5.4 GB/s for large data sets. This implementation is highly effective for legacy compatibility but may be less desirable for systems where absolute maximum throughput is the primary requirement.

6 Comparative Analysis

According to the results obtained, we can see that SHA-3 is the fastest algorithm across all the tested categories. Its peak performance reached an impressive average of 6,754.36 MB/s for large files, likely benefiting from modern instruction set optimizations.

All algorithms showed near-linear scaling as the file size increases. For example, jumping from 10MB to 100MB resulted in a 10x increase in throughput (e.g., Blake2b went from 557 MB/s to 5608 MB/s), which indicates that they are highly efficient and effectively utilize system memory and cache.

For the 1KB file, the throughput is extremely low (approximately 0.06 MB/s). This is expected, as the average execution time (14-29 ms) is dominated by the operating

system overhead of starting a new process and initializing the OpenSSL environment, rather than the actual mathematical hashing.

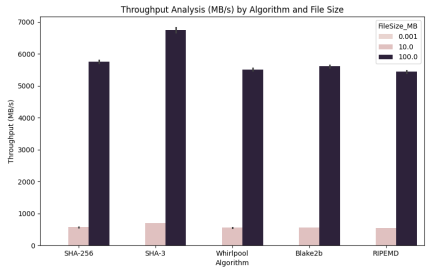

Fig. 3
Throughput

SHA-3 also showed the most consistent results for small files, with a standard deviation of only 0.58 ms. In contrast, SHA-256 had a massive outlier in its first iteration (1,197 ms), which inflated its average response time significantly. RIPEMD is clearly the slowest of the five, with the performance gap between RIPEMD and SHA-3 being approximately 1.3 GB/s.

Based on the results, we can rank the implementations by their efficiency under heavy load (100 MB), based on their throughout values, from best to worst: SHA-3, SHA-256, Blake2b, Whirlpool, RIPEMD.

Regarding the time consumption, we must distinguish between total time, which includes system overhead, and processing time. Initialization Overhead: For the 1 KB file, all implementations took between 14 ms and 29 ms. Since the actual hashing of 1 KB takes less than 0.001 ms, this shows that 99 percent of the time recorded for small files is system overhead (loading the .exe and OpenSSL). It's important to note that for SHA-3, processing 100 MB took almost exactly the same time as 1 KB. This proves the implementation is bottlenecked by the CPU's ability to start the process, not by the complexity of the math.

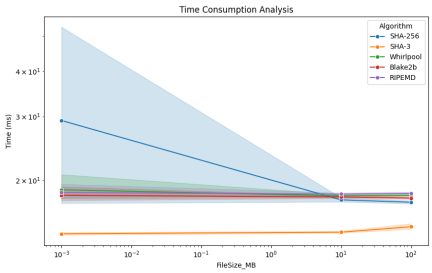

Fig. 4
Time Consumption

The security of a hash function is highly determined by the collision resistance of the function. Out of the 5 function we tested, it it known that SHA-3, BLAKE2b and Whirlpool are resistant, while SHA-256 and RIPEMD-160 are more vulnerable to this.

7 Conclusion

Based on all the tested iterations, SHA-3 is the recommended implementation for high-throughput environments, achieving a peak speed of 6.75 GB/s. However, BLAKE2b provides the most consistent timing results with a standard deviation of less than 0.7 ms, making it ideal for systems where timing predictability is critical. SHA-256 remains a strong secondary choice, particularly for larger files where its initial startup latency is mitigated.

Based on the results, it would seem that overall SHA-3 and BLAKE3b present the most impressive result. However, each algorithm can be a good choice, depending on the desired result and critical aspects, and as such they have somewhat different areas of usage.

SHA-256 can be used for public web services (SSL/TLS), Bitcoin mining, and standard software signing (Windows/Linux installers), as it it known to be a highly compatible hash function.

SHA-3 can be used in high-security government applications or next-generation blockchain protocol, as it is the newest NIST standard and has the highest level of theoretical security and immunity to all known structural attacks. The result we obtained also prove that it is the fastest, making it a great choice for high-end systems.

BLAKE2b can be used for cloud storage,high-speed file systems or real-time integrity checks for large backups. Our data proved that this hashing algorithem would provide hight stability regardless of the size of data.

Whirlpool can be used for disk encryption headers or "cascading" hashes (hashing the hash of another algorithm), as it is known for its significant robust.

RIPEMD-160 can be used for Bitcoin address generation or PGP (Pretty Good Privacy) signatures., as it produces a shorter 160-bit hash. This is ideal for situations where storage space is limited or a shorter, human-readable ID in needed.

Future research can extend beyond software analyses that I did so far. Most current hash functions (like SHA-256 and RIPEMD) are known to be vulnerable to future quantum computer attacks using Grover's Algorithm, which effectively halves their security bit-length. So post-Quantum Cryptographic (PQC) integration would a good future testing parameter to investigate, to determine the performance impact of transitioning to quantum-resistant algorithms (like those currently being standardized by NIST).

Also, while this paper focuses on software-level measurements, physical hardware often leaks information through power consumption or electromagnetic emissions. Because of this, another future research opportunity would be to conduct a comparative study on the power-efficiency and side-channel leakage of these five algorithms.

References

[1] Tiwari, A.: Distributed computing to blockchain. In: Rajiv Pandey, S.G., Fatima,

S. (eds.) Distributed Computing to Blockchain, pp. 251–265. Academic Press, ??? (2023). Chap. Chapter 14 - Cryptography in blockchain

[2] Johnson, R.: Hash Functions and Algorithms: Definitive Reference for Developers and Engineers. NOBTREX, ??? (2025)

[3] Sevin, A.: Implementation of a Data-Parallel Approach on a Lightweight Hash Function for IoT Devices. MDPI Mathematics 2025 **13**(5) (2025) https://doi.org/10.3390/math13050734

[4] Hariharasubramanian, N.: What Is Hashing in Cyber Security and Why It Matters. [Online]. Available: https://fidelissecurity.com/cybersecurity-101/learn/what-is-hashing/ (2025)

[5] eitca: How does a hash function ensure data integrity and security? [Online]. Available: https://eitca.org/cybersecurity/eitc-is-acc-advanced-classical-cryptography/hash-functions/introduction-to-hash-functions/examination-review-introduction-to-hash-functions/how-does-a-hash-function-ensure-data-integrity-and-security/ (2023)

[6] rain: Hash. [Online]. Available: https://www.rain.com/glossary/hash (2026)

[7] Sapkota, S.: Blockchain technology in modern days and future. Master's thesis, UNIVERSITY OF VAASA (2025)

[8] Yadav, J.Y., Shevkar, R.S.: Exploring Efficacy of Cryptographic Hash Functions SHA-256 and BLAKE2s Against HashBoost. Paper presented at the International Conference on Advanced Network Technologies and Intelligent Computing, in 2024, First online on 08 March 2025 (2025)

[9] Santiago, L.: Sponge Constructions Explained: From Keccak to Poseidon. [Online]. Available: https://medium.com/@lffsssantiago/sponge-constructions-explained-from-keccak-to-poseidon-5d166e1c9e07 (2025)

[10] Chen, S., Guo, J., List, E., Shi, D., Zhang, T.: Scrutinizing the Security of AES-based Hashing and One-way Functions. Cryptology ePrint Archive, Paper 2025/792 (2025). https://eprint.iacr.org/2025/792

[11] Sugier, J.: Scrutinizing the Security of AES-based Hashing and One-way Functions (2024). https://doi.org/10.1109/DESSERT65323.2024.11122172